

We might install other related packages, if required, as we proceed. If you are encountering problems at any stage of the installation, please refer to the following link: <https://packaging.python.org/tutorials/installing-packages/>.

Note Refer to the Python package index, PyPI (<https://pypi.python.org/pypi>), to search for the latest packages available.

Follow the steps to install pip via <https://pip.pypa.io/en/stable/installing/>.

Python Packages

We will be covering the references to the installation steps and the initial-level coding for the Pandas, NumPy, and SciPy packages. Currently, Python offers versions 2.x and 3.x, with compatible functions for machine learning. We will be making use of Python2.7 and Python3.5, where required. Version 3.5 has been used extensively throughout the chapters of this book.

NumPy

NumPy is used particularly for scientific computing in Python. It is designed to efficiently manipulate large multidimensional arrays of arbitrary records, without sacrificing too much speed for small multidimensional arrays. It could also be used as a multidimensional container for generic data. The ability of NumPy to create arrays of arbitrary type, which also makes NumPy suitable for interfacing with general-purpose data-base applications, makes it one of the most useful libraries you are going to use throughout this book, or thereafter for that matter.

Following are the codes using the NumPy package. Most of the lines of code have been appended with a comment, to make them easier to understand by the user.

```
## Numpy

import numpy as np          # Importing the Numpy package
a= np.array([1,4,5,8], float) # Creating Numpy array with
                               # Float variables
print(type(a))              #Type of variable
> <class 'numpy.ndarray'>

# Operations on the array
a[0] = 5                    #Replacing the first element of the array
print(a)
> [ 5.  4.  5.  8.]

b = np.array([[1,2,3],[4,5,6]], float) # Creating a 2-D numpy
                                         # array
b[0,1]                        # Fetching second element of 1st array
> 2.0

print(b.shape)               #Returns tuple with the shape of array
> (2, 3)

b.dtype                      #Returns the type of the value stored
> dtype('float64')

print(len(b))                #Returns length of the first axis
> 2

2 in b                       #'in' searches for the element in the array
> True

0 in b
> False
```

```
# Concatenation of 2 or more arrays
m = np.array([1,2], float)
n = np.array([3,4,5,6], float)
p = np.concatenate((m,n))
print(p)
> [ 1.  2.  3.  4.  5.  6.]
(6,)

print(p.shape)

# 'newaxis' : to increase the dimensionality of the array
q = np.array([1,2,3], float)
q[:, np.newaxis].shape
> (3, 1)
```

NumPy has other functions, such as `zeros`, `ones`, `zeros_like`, `ones_like`, `identity`, `eye`, which are used to create arrays filled with 0s, 1s, or 0s and 1s for given dimensions.

Addition, subtraction, and multiplication occur on same-size arrays.

Multiplication in NumPy is offered as element-wise and not as matrix multiplication. If the arrays do not match in size, the smaller one is repeated to perform the desired operation. Following is an example for this:

```
a1 = np.array([[1,2],[3,4],[5,6]], float)
a2 = np.array([-1,3], float)
print(a1+a2)
> [[ 0.  5.] [ 2.  7.] [ 4.  9.]]
```


Note `pi` and `e` are included as constants in the NumPy package.

One can refer to the following sources for detailed tutorials on NumPy: www.numpy.org/ and <https://docs.scipy.org/doc/numpy-dev/user/quickstart.html>.

NumPy offers few of the functions that are directly applicable on the arrays: `sum` (summation of elements), `prod` (product of the elements), `mean` (mean of the elements), `var` (variance of the elements), `std` (standard deviation of the elements), `argmin` (index of the smallest element in array), `argmax` (index of the largest element in array), `sort` (sort the elements), `unique` (unique elements of the array).

```
a3 = np.array([[0,2],[3,-1],[3,5]], float)
print(a3.mean(axis=0))           # Mean of elements column-wise
> [ 2.  2.]

print(a3.mean(axis=1))          # Mean of elements row-wise
> [ 1.  1.  4.]
```

Note To perform the preceding operations on a multidimensional array, include the optional argument `axis` in the command.

NumPy offers functions for testing the values present in the array, such as `nonzero` (checks for nonzero elements), `isnan` (checks for “not a number” elements), and `isfinite` (checks for finite elements). The `where` function returns an array with the elements satisfying the following conditions:

```
a4 = np.array([1,3,0], float)
np.where(a!=0, 1/a ,a)
> array([ 0.2 , 0.25 , 0.2 , 0.125])
```

To generate random numbers of varied length, use the random function from NumPy.

```
np.random.rand(2,3)
> array([[ 0.41453991, 0.46230172, 0.78318915],
[0.54716578, 0.84263735, 0.60796399]])
```

Note The random number seed can be set via `numpy.random.seed (1234)`. NumPy uses the Mersenne Twister algorithm to generate pseudorandom numbers.

Pandas

Pandas is an open sourced software library. DataFrames and Series are two of its major data structures that are widely used for data analysis purposes. Series is a one-dimensional indexed array, and DataFrame is tabular data structure with column- and row-level indexes. Pandas is a great tool for preprocessing datasets and offers highly optimized performance.

```
import pandas as pd
series_1 = pd.Series([2,9,0,1])      # Creating a series object
print(series_1.values)               # Print values of the
                                     series object

> [2 9 0 1]

series_1.index                       # Default index of the series object
> RangeIndex(start=0, stop=4, step=1)

series_1.index = ['a','b','c','d']   #Settnig index of the
                                     series object

series_1['d']                         # Fetching element using new index
> 1
```

```
# Creating dataframe using pandas
class_data = {'Names': ['John', 'Ryan', 'Emily'],
              'Standard': [7, 5, 8],
              'Subject': ['English', 'Mathematics', 'Science']}
class_df = pd.DataFrame(class_data, index = ['Student1',
      'Student2', 'Student3'],
                        columns = ['Names', 'Standard', 'Subject'])

print(class_df)
>
      Names  Standard  Subject
Student1  John       7    English
Student2  Ryan       5  Mathematics
Student3  Emily      8     Science

class_df.Names
>Student1    John
Student2    Ryan
Student3    Emily
Name: Names, dtype: object

# Add new entry to the dataframe
import numpy as np
class_df.ix['Student4'] = ['Robin', np.nan, 'History']
class_df.T      # Take transpose of the dataframe
>
      Student1  Student2  Student3  Student4
Names      John      Ryan      Emily      Robin
Standard    7         5         8         NaN
Subject  English  Mathematics  Science   History
```

The following code is used to change the Datatype of the column to a “category” type:

```
# Changing datatype of the column
class_data['Grade'] = class_data['Grade'].astype('category')
class_data.Grade.dtypes
> category
```

The following stores the results to a .csv file:

```
# Storing the results
class_data.to_csv('class_dataset.csv', index=False)
```

Among the pool of functions offered by the Pandas library, merge functions (concat, merge, append), groupby, and pivot_table functions have an intensive application in data processing tasks. Refer to the following source for detailed Pandas tutorials: <http://pandas.pydata.org/>.

SciPy

SciPy offers complex algorithms and their use as functions in NumPy. This allocates high-level commands and a variety of classes to manipulate and visualize data. SciPy is curated in the form of multiple small packages, with each package targeting individual scientific computing domains. A few of the subpackages are linalg (linear algebra), constants (physical and mathematical constants), and sparse (sparse matrices and associated routines).

Most of the NumPy package functions applicable on arrays are also included in the SciPy package. SciPy offers pre-tested routines, thereby saving a lot of processing time in the scientific computing applications.

```
import scipy
import numpy as np
```


Note SciPy offers in-built constructors for objects representing random variables.

Following are a few examples from Linalg and Stats out of multiple subpackages offered by SciPy. As the subpackages are domain-specific, it makes SciPy the perfect choice for data science.

SciPy subpackages, here for linear algebra (`scipy.linalg`), are supposed to be imported explicitly in the following manner:

```
from scipy import linalg
mat_ = np.array([[2,3,1], [4,9,10], [10,5,6]]) # Matrix Creation
print(mat_)
> [[ 2 3 1] [ 4 9 10] [10 5 6]]

linalg.det(mat_)           # Determinant of the matrix
inv_mat = linalg.inv(mat_) # Inverse of the matrix
print(inv_mat)
> [[ 0.02409639 -0.07831325 0.12650602] [ 0.45783133 0.01204819
-0.09638554] [-0.42168675 0.12048193 0.03614458]]
```

The code for performing singular value decomposition and storing the individual components follows:

```
# Singular Value Decomposition
comp_1, comp_2, comp_3 = linalg.svd(mat_)
print(comp_1)
print(comp_2)
print(comp_3)
```